4.1 Configuration in Spring: Properties and profiles

The term *configuration* can have different meanings depending on the context. When discussing the Spring Framework's core features and its `ApplicationContext`, configuration refers to which beans (Java objects registered in Spring) have been defined to be managed by the Spring container and are injected where needed. For example, you can define beans in an XML file (XML configuration), in a `@Configuration` class (Java configuration), or by relying on annotations like `@Component` (annotation-driven configuration).

In this book, unless specified otherwise, whenever I mention *configuration*, I don't mean the previous concept, but rather everything that's likely to change between deployments, as defined by the 15-Factor methodology.

Spring provides you with a convenient `Environment` abstraction that gives you access to any configuration data, no matter its source. The two key aspects of a Spring application environment are *properties* and *profiles*. You have already worked with properties in the previous chapter. Profiles are a tool for labeling logical groups of beans or configuration data that should be loaded at runtime only if a given profile is enabled. Figure 4.2 shows the main aspects of a Spring application environment.